NURobotics Firmware Course

Week 4: Driver Development

Course Overview

- Week 1: Intro to Firmware & Programming
- Week 2: Analog I/O & Interrupts
- Week 3: Serial Communication Protocols
- **Week 4: Driver Development**
- Week 5: State Machines + RTOSes
- Week 6: STM32 Fundamentals
- Week 7: STM32 Peripherals
- Week 8: Communication and RTOS Capstone

Drivers

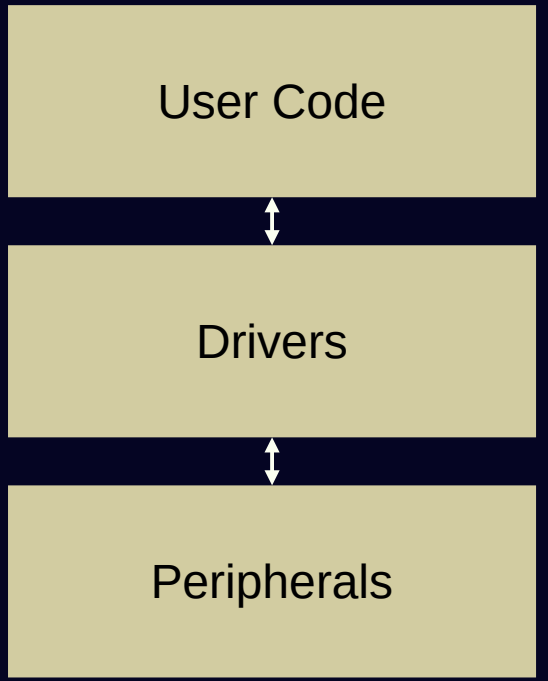
What is a Driver

A driver is a layer of abstraction between a physical device and the software using it.

A driver provides:

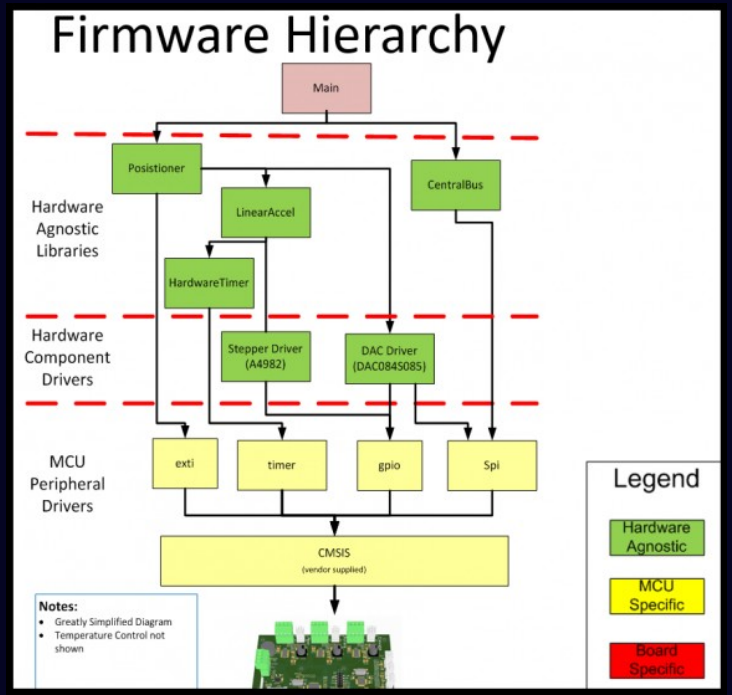
- Functions to initialize and control the device
- Relevant data and data structures to use the device
- Basic informmation on how the device works

The goal is to make using the device as simple as posible for the layer above



Abstraction Layers

- Hardware
  - Registers and memory-mapped peripherals
- Hardware Abstraction Layer (HAL)
  - Memory access simplification
  - Peripheral access simplification
  - Often provided by hardware manufacturer
  - *(volatile uint32_t *)0x40020014 |= (1 << 14); → digitalWrite(14, HIGH)
- Drivers
  - Sometimes use HAL, sometimes don’t
  - Wrap primitive peripherals into actual devices
  - Can sometimes be found online (GitHub, etc.)
  - i2c_write(0x3C, buf, len) → display_drawText("Hi", 0, 0)
- User Level Program
  - Combines drivers to produce desired behavior
  - Rarely touches lower layers directly



Macros

- Preprocessor directives that replace code before the compilation process
- Commonly used to define constants and reusable code snippets

```
#define LED_PIN 13

void setup() {

  pinMode(LED_PIN, OUTPUT);

}
```

```
#define SQUARE(x) (x * x)
#define PRINT(x) Serial.println(x)

void setup() {

  uint8_t value = SQUARE(2);
  PRINT(value);

}
```

Macros

```
#ifndef IMU_HPP
#define IMU_HPP

/**
 * --- CODE ---
 */

#endif
```

```
#include <Arduino.h>

void setup() {

  #ifdef Arduino_h
  uint8_t value = 32;
  #else
  uint8_t value = 100;
  #endif

}
```

Enums

- User-defined data type that assigns meaningful names to a set of integer constants
- By default, the first enum constant has the value 0, and each subsequent constant is incremented by 1

```
enum Direction {
  NORTH,
  EAST,
  SOUTH,
  WEST,
};
```

```
enum ImuError {
  // No error was detected during operation
  NO_ERROR = 0,
  // The IMU did not respond to the provided address
  I2C_WRONG_ADDRESS = 1,
  // The IMU was unable to be initialized properly
  IMU_INIT_ERROR=2,
};
```

Reading 101

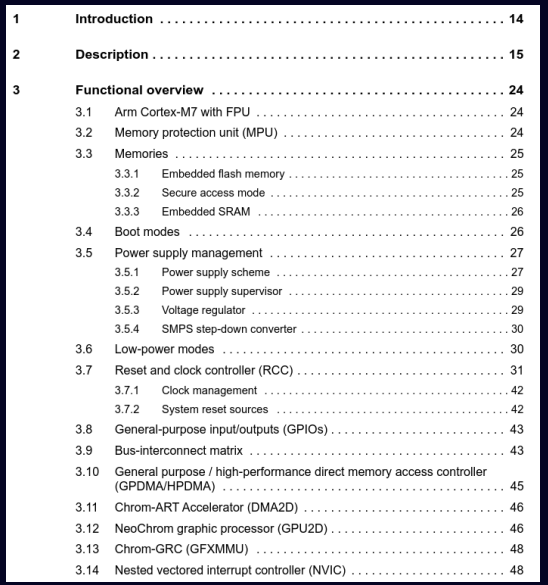
Technical Documents

- Datasheets
  - A high-level overview of the device
  - Describes
    - Pins
    - Use cases
    - Basic layouts
    - Some registers
- Technical Reference Manuals (TRMs)
  - Complete documentation of the device
  - More in-depth than datasheets
  - Primarily relevant for MCUs
- Hardware Reference Manual
  - Gives in-depth application examples
  - Outlines specific electrical expectations for the device

Reading Datasheets

The STM32H7 datasheet is 325 pages. This is far from the largest datasheet you will come across. However, if you know what to look for and how to look for it, you won't need most of it!

The table of contents is the best starting point.



1 Introduction . . . 14

2 Description . . . 15

3 Functional overview . . . 24

3.1 Arm Cortex-M7 with FPU . . . 24
3.2 Memory protection unit (MPU) . . . 24
3.3 Memories . . . 25
3.3.1 Embedded flash memory . . . 25
3.3.2 Secure access mode . . . 25
3.3.3 Embedded SRAM . . . 26
3.4 Boot modes . . . 26
3.5 Power supply management . . . 27
3.5.1 Power supply scheme . . . 27
3.5.2 Power supply supervisor . . . 29
3.5.3 Voltage regulator . . . 29
3.5.4 SMPS step-down converter . . . 30
3.6 Low-power modes . . . 30
3.7 Reset and clock controller (RCC) . . . 31
3.7.1 Clock management . . . 42
3.7.2 System reset sources . . . 42
3.8 General-purpose input/outputs (GPIOs) . . . 43
3.9 Bus-interconnect matrix . . . 43
3.10 General purpose / high-performance direct memory access controller (GPDMA/HPDMA) . . . 45
3.11 Chrom-ART Accelerator (DMA2D) . . . 46
3.12 NeoChrom graphic processor (GPU2D) . . . 46
3.13 Chrom-GRC (GFXMMU) . . . 48
3.14 Nested vectored interrupt controller (NVIC) . . . 48

Technical Documentation Important Sections

- Register Maps
  - Outlines what each register is for
- Pin Maps
  - Maps pins to uses
- Usage outlines
  - Goes over the expected usage of a certain functionality
- Addresses/Value selects
  - Usually scattered throughout the sheet near their relevant sections

Lab!

Today’s Lab

- IMU driver
  - Complete the frameworked files to create a functional driver for the IMU
  - Use the IMU’s datasheet to accomplish this